

AI-SHA Farm Brain

Fully Autonomous Agricultural Robot Orchestrator Node

Technical Documentation

Platform: NVIDIA Jetson Orin Nano 8GB

Framework: ROS 2 Humble

Navigation: Nav2 + Isaac Sim RL Policy

Vision: YOLOv8m (TensorRT) + MobileNetV3 Plant Disease

Sensors: ~16 environmental + 2 linear actuators

Repository: github.com/Ahmed28309/AI-SHA

Table of Contents

1. System Overview
2. Architecture
 - 2.1 State Machine
 - 2.2 Navigation Stack
 - 2.3 Isaac Sim Integration
 - 2.4 Obstacle Avoidance
3. Sensor Suite (~16 Sensors)
4. Actuators
 - 4.1 Linear Actuators (Soil Probes)
 - 4.2 Water Pump & Seed Dispenser
5. ROS 2 Node Details
 - 5.1 Subscribed Topics
 - 5.2 Published Topics
 - 5.3 Parameters
6. Autonomous Decision Engine
7. Voice Override (Secondary)
8. Waypoint Sequence
9. Configuration
10. Launch
11. Data Logging
12. All ROS 2 Nodes in the System

1 System Overview

AI-SHA is a fully autonomous agricultural robot built on three compute platforms: an NVIDIA Jetson Orin Nano 8GB (AI processing), a Raspberry Pi 5 (display and TTS), and a Raspberry Pi 4 (motor control, encoders, and low-level sensors).

The Farm Brain node (`farm_brain.py`) is the central orchestrator running on the Jetson. It operates autonomously by default -- the robot patrols pre-defined farm sections, deploys soil probes via linear actuators, reads ~16 environmental sensors, detects plant diseases using computer vision, and takes corrective actions such as watering dry soil.

Human voice commands serve only as an override channel. The robot does not require human input to begin or continue its work. Voice commands like "stop", "pause", "skip", or "go to row 3" override the autonomous loop when spoken.

The three core intelligence layers are:

1. LLM (Llama 3.2 3B) -- parses natural language into intents
2. Isaac Sim RL Policy -- trained navigation / obstacle avoidance
3. Farm Brain State Machine -- autonomous decision-making

2 Architecture

The system is distributed across three boards connected via Ethernet (ROS 2 DDS). Each board runs its own set of ROS 2 nodes.

Jetson Orin Nano (ROS 2 Humble)

```
+-- farm_brain      (orchestrator, this node)
+-- robot_brain     (LLM text generation)
+-- llm_node        (Gemini / Llama 3.2)
+-- stt_node        (Faster-Whisper GPU)
+-- yolov8_ros      (YOLOv8m TensorRT + plant disease)
+-- realsense2_camera (RGB-D)
+-- ldlidar_stl_ros2 (LD-19 LiDAR)
+-- slam_toolbox    (SLAM / localization)
```

Raspberry Pi 5 (ROS 2 Jazzy)

```
+-- tts_elevenlabs  (ElevenLabs TTS)
+-- llm_display      (robot face + dashboard)
+-- speaker_monitor  (audio state)
+-- bno055_imu       (9-DOF IMU)
```

Raspberry Pi 4 (ROS 2 Humble)

```
+-- mecanum_driver  (4x motor control)
+-- motor_control   (encoders, odometry)
+-- soil_moisture    (Arduino serial)
+-- rain_sensor      (Arduino serial)
+-- bmp180_pressure  (I2C)
+-- gps_gt_u7        (serial NMEA)
+-- dht11_node       (GPIO)           [placeholder]
+-- ph_sensor_node   (Arduino ADC)     [placeholder]
+-- uv_sensor_node    (I2C / analog)   [placeholder]
+-- lux_sensor_node   (I2C)            [placeholder]
+-- gas_sensor_node   (Arduino ADC)     [placeholder]
```

2.1 State Machine

The Farm Brain operates as a finite state machine with the following states:

State	Description
STARTUP	Wait for sensors to come online (~10s)
IDLE	Between patrols; check if patrol interval elapsed
PATROLLING	Iterating through patrol waypoint list
NAVIGATING	Travelling to a waypoint via Nav2 or fallback
DEPLOYING_PROBES	Extending linear actuators into soil
MEASURING	Waiting for sensor readings to stabilise
INSPECTING	Vision scan for plant diseases
ANALYSING	Evaluating sensor data against thresholds
WATERING	Pump activated for dry soil
SOWING	Seed dispenser activated

RETRACTING_PROBES	Pulling actuators out of soil
RETURNING_HOME	Navigating back to home/charger
PAUSED	Human-requested pause (voice override)
EMERGENCY_STOP	Obstacle too close or fault detected
ERROR	Recoverable error state

Primary autonomous flow:

STARTUP -> IDLE -> PATROLLING -> NAVIGATING -> DEPLOYING_PROBES -> MEASURING -> INSPECTING -> RETRACTING_PROBES -> ANALYSING -> [WATERING | SOWING] -> (next waypoint) -> ... -> RETURNING_HOME -> IDLE

2.2 Navigation Stack

Navigation uses a layered approach:

Layer 1 -- Nav2 (primary): ROS 2 Navigation2 stack provides global path planning (NavFn / Smac) and local trajectory planning (DWB / MPPI) using the SLAM-generated costmap. The farm brain sends NavigateToPose goals via the Nav2 action server.

Layer 2 -- Isaac Sim RL Policy: A reinforcement learning policy trained in NVIDIA Isaac Sim provides learned obstacle avoidance and locomotion. The policy takes a downsampled LiDAR scan (36 bins), IMU data (6 DOF), and relative goal position (3) as input, and outputs linear and angular velocity commands. This runs as a local planner complement or fallback.

Layer 3 -- Reactive safety: A 10 Hz obstacle check monitors LiDAR minimum range and RealSense forward depth. If an obstacle is closer than 0.30 m, an emergency stop is triggered. Between 0.30-0.80 m, speed is reduced.

2.3 Isaac Sim Integration

The Isaac Sim integration follows a sim-to-real transfer pipeline:

1. Train an RL agent in Isaac Sim using the Omniverse environment (e.g. Isaac Gym / Orbit). The agent learns to navigate to goals while avoiding obstacles using LiDAR + IMU observations.
2. Export the trained policy as ONNX or TensorRT engine.
3. Load the engine in farm_brain.py via `_load_isaac_model()`.
4. At runtime, `_build_observation()` constructs the observation vector from live sensor data, and `_query_isaac_policy()` runs inference to produce velocity commands.

Observation vector structure (42 dimensions):

[lidar_36_bins, accel_x, accel_y, accel_z, rel_goal_x, rel_goal_y, goal_dist]

Action vector (2 dimensions):

[linear_velocity_x, angular_velocity_z]

2.4 Obstacle Avoidance

Obstacle avoidance runs continuously at 10 Hz (`_obstacle_check`) regardless of the autonomous state. It monitors two sources:

- LiDAR (LD-19): 360-degree 2D scan, minimum valid range extracted
- RealSense D435: Forward depth minimum (published as Float32)

The nearest obstacle distance triggers three behaviours:

1. > 0.80 m: normal speed
2. $0.30 - 0.80$ m: reduced speed (Isaac Sim policy can fine-tune)
3. < 0.30 m: EMERGENCY STOP -- zero velocity, alert published

When the obstacle clears, the robot automatically resumes navigation.

3 Sensor Suite (~16 Sensors)

The farm brain subscribes to all sensor topics and aggregates them into a SensorSnapshot dataclass. Sensor data is published as a JSON summary on /farm_brain/sensor_summary every 10 seconds.

#	Sensor	Interface	ROS Topics	Data
1	Soil Moisture	Arduino Serial	/soil_moisture/*	Moisture %, raw ADC, dry bool
2	pH Sensor	Arduino ADC	/ph_sensor/*	pH value (0-14), raw ADC
3	BMP180	I2C (RPi)	/bmp180/*	Temp (C), Pressure (Pa), Alt (m)
4	DHT11	GPIO (RPi)	/dht11/*	Temperature (C), Humidity (%)
5	BNO055 IMU	I2C (RPi)	/imu/data	Quaternion, Accel, Gyro, Mag
6	UV Sensor	I2C / Analog	/uv_sensor/*	UV index, raw ADC
7	Lux Sensor	I2C	/lux_sensor/lux	Ambient light (lux)
8	Gas / CO2 (MQ-135)	Arduino ADC	/gas_sensor/*	CO2 ppm, raw ADC
9	Raindrop Sensor	Arduino Serial	/rain_sensor/*	Intensity %, raining bool
10	GPS (GT-U7)	Serial NMEA	/gps/fix	Lat, Lon, Alt, Fix status
11	Encoder FL	GPIO IRQ	/encoders/fl_rpm	RPM (600 PPR)
12	Encoder FR	GPIO IRQ	/encoders/fr_rpm	RPM (600 PPR)
13	Encoder RL	GPIO IRQ	/encoders/rl_rpm	RPM (600 PPR)
14	Encoder RR	GPIO IRQ	/encoders/rr_rpm	RPM (600 PPR)
15	LiDAR (LD-19)	Serial	/scan	360 deg, ~4000 pts, 10 Hz
16	RealSense D435	USB	/camera/depth/*	RGB 640x480, Depth 848x480

4 Actuators

4.1 Linear Actuators (Soil Probes)

Two linear actuators are used to insert soil moisture and pH probes into the ground at each waypoint. The actuators are controlled via Int32 commands on dedicated topics:

/actuators/linear_actuator_left (+1=extend, -1=retract, 0=stop)

/actuators/linear_actuator_right (+1=extend, -1=retract, 0=stop)

The deploy/retract durations are configurable parameters (default 3 seconds each). On emergency stop or node shutdown, probes are automatically retracted for safety.

4.2 Water Pump & Seed Dispenser

Water pump: /actuators/water_pump (Bool: true=on, false=off)

Seed dispenser: /actuators/seed_dispenser (Bool: true=on, false=off)

The water pump activates for water_duration_sec (default 8s) when soil moisture falls below thresh_soil_dry_pct (default

30%) and it is not raining. The seed dispenser activates for `sow_duration_sec` (default 4s) when requested via voice override or when autonomous sowing logic is triggered.

Actuator	Topic	Message Type	Values
Linear Act. Left	/actuators/linear_actuator_left	std_msgs/Int32	+1=extend, -1=retract, 0=stop
Linear Act. Right	/actuators/linear_actuator_right	std_msgs/Int32	+1=extend, -1=retract, 0=stop
Water Pump	/actuators/water_pump	std_msgs/Bool	true=on, false=off
Seed Dispenser	/actuators/seed_dispenser	std_msgs/Bool	true=on, false=off
Mecanum Motors	/cmd_vel	geometry_msgs/Twist	linear.x, angular.z

5 ROS 2 Node Details

Node name: farm_brain

Package: robot_brain

Entry point: farm_brain = robot_brain.farm_brain:main

5.1 Subscribed Topics (~35 subscriptions)

Topic	Type	Source
/soil_moisture/moisture	Float32	RPi4 soil_moisture node
/soil_moisture/dry	Bool	RPi4 soil_moisture node
/soil_moisture/raw	Int32	RPi4 soil_moisture node
/ph_sensor/ph	Float32	RPi4 ph_sensor node
/ph_sensor/raw	Int32	RPi4 ph_sensor node
/bmp180/temperature	Temperature	RPi4 bmp180 node
/bmp180/pressure	FluidPressure	RPi4 bmp180 node
/bmp180/altitude	Float64	RPi4 bmp180 node
/dht11/temperature	Float32	RPi4 dht11 node
/dht11/humidity	Float32	RPi4 dht11 node
/imu/data	Imu	RPi5 bno055 node
/uv_sensor/index	Float32	RPi4 uv_sensor node
/uv_sensor/raw	Int32	RPi4 uv_sensor node
/lux_sensor/lux	Float32	RPi4 lux_sensor node
/gas_sensor/co2_ppm	Float32	RPi4 gas_sensor node
/gas_sensor/raw	Int32	RPi4 gas_sensor node
/rain_sensor/intensity	Float32	RPi4 rain_sensor node
/rain_sensor/raining	Bool	RPi4 rain_sensor node
/rain_sensor/raw	Int32	RPi4 rain_sensor node
/gps/fix	NavSatFix	RPi4 gps_gt_u7 node
/encoders/fl_rpm	Float32	RPi4 motor_control node
/encoders/fr_rpm	Float32	RPi4 motor_control node
/encoders/rl_rpm	Float32	RPi4 motor_control node
/encoders/rr_rpm	Float32	RPi4 motor_control node
/odom	Odometry	RPi4 motor_control node
/scan	LaserScan	Jetson LiDAR driver
/camera/depth/min_forward	Float32	Jetson depth processor
/detection/objects_simple	String	Jetson yolov8_ros
/detection/disease_simple	String	Jetson yolov8_ros
/speech/text	String	Jetson stt_node (voice)
/farm_brain/intent	String	Jetson LLM (parsed JSON)

5.2 Published Topics

Topic	Type	Rate	Description
/cmd_vel	Twist	10 Hz	Velocity commands (fallback nav)
/tts_text	String	Event	Spoken announcements to TTS
/farm_brain/status	String	0.33 Hz	JSON: state, waypoint, progress
/farm_brain/sensor_summary	String	0.1 Hz	JSON: all sensor readings
/farm_brain/alerts	String	Event	JSON: type, message, waypoint
/farm_brain/measurement	String	Event	JSON: per-waypoint measurement
/actuators/water_pump	Bool	Event	Pump on/off
/actuators/seed_dispenser	Bool	Event	Seeder on/off
/actuators/linear_actuator_left	Int32	Event	Probe extend/retract/stop
/actuators/linear_actuator_right	Int32	Event	Probe extend/retract/stop

5.3 Parameters

Parameter	Default	Description
auto_start	true	Begin autonomous patrol on startup
nav2_enabled	true	Use Nav2 for navigation
obstacle_avoidance_enabled	true	Reactive obstacle layer
isaac_model_path	"	Path to TensorRT/ONNX RL policy
farm_locations_file	"	JSON file with locations + patrol order
patrol_interval_sec	300.0	Seconds between patrol cycles
probe_deploy_time_sec	3.0	Linear actuator extend time
probe_retract_time_sec	3.0	Linear actuator retract time
measure_settle_time_sec	5.0	Sensor stabilisation wait
water_duration_sec	8.0	Pump activation time per waypoint
sow_duration_sec	4.0	Seed dispenser activation time
inspect_duration_sec	5.0	Vision scan duration
thresh_soil_dry_pct	30.0	Soil moisture % trigger for watering
thresh_ph_low	5.5	Acidic soil alert threshold
thresh_ph_high	8.0	Alkaline soil alert threshold
thresh_co2_high_ppm	1000.0	CO2 ppm alert threshold
thresh_uv_high_index	8.0	UV index alert threshold
thresh_temp_high_c	40.0	Heat warning (Celsius)
thresh_temp_low_c	5.0	Frost warning (Celsius)
thresh_obstacle_stop_m	0.30	Emergency stop distance
thresh_obstacle_slow_m	0.80	Slow-down distance
thresh_disease_conf_min	0.60	Min disease detection confidence
log_file	farm_brain_log.json	Patrol log output file

6 Autonomous Decision Engine

At each patrol waypoint, after measuring, the brain evaluates sensor data against configurable thresholds and takes automatic actions:

Condition	Threshold	Action
Soil moisture < thresh	< 30%	Activate water pump (unless raining)
pH too low	< 5.5	Publish acidic soil alert
pH too high	> 8.0	Publish alkaline soil alert
CO2 elevated	> 1000 ppm	Publish CO2 alert
Temperature high	> 40 C	Publish heat warning
Temperature low	< 5 C	Publish frost warning
UV index high	> 8.0	Publish UV warning
Disease detected	conf > 60%	Log disease + publish alert
Obstacle close	< 0.30 m	Emergency stop (continuous 10Hz)

All alerts are published on `/farm_brain/alerts` as JSON with fields: type, message, timestamp, waypoint. Measurements at each waypoint are logged to disk and published on `/farm_brain/measurement`.

7 Voice Override (Secondary)

Voice commands are a secondary override channel. The robot does not require voice input to operate. When a human speaks, the STT node publishes text on `/speech/text`, and the farm brain parses it as a simple override:

Voice Command	Action	Effect
"Stop" / "Halt"	stop	Emergency stop, cancel all motion
"Pause" / "Wait"	pause	Pause autonomous loop
"Resume" / "Continue"	resume	Resume from paused state
"Skip" / "Next"	skip	Skip current waypoint, advance
"Report" / "Status"	report	Speak full sensor summary via TTS
"Go to row 1"	go_to	Navigate to named location
"Water"	water	Force watering at current position
"Sow" / "Plant"	sow	Force sowing at current position
"Home" / "Return"	go_to home	Navigate to home position

8 Waypoint Sequence

At each patrol waypoint, the farm brain executes the following sequence in a dedicated thread:

1. DEPLOYING_PROBES -- Extend linear actuators into soil
wait probe_deploy_time_sec (3s)
2. MEASURING -- Wait measure_settle_time_sec (5s)
Take SensorSnapshot (all ~16 sensors)

3. INSPECTING	-- Wait inspect_duration_sec (5s) YOLOv8 + plant disease classifier scan
4. RETRACTING_PROBES	-- Retract linear actuators Wait probe_retract_time_sec (3s)
5. ANALYSING	-- Compare readings against thresholds Trigger actions (water, alerts)
6. [WATERING]	-- If soil dry: pump for water_duration_sec
7. LOG	-- Save WaypointMeasurement to patrol_log Publish on /farm_brain/measurement
8. ADVANCE	-- Move patrol_index to next waypoint

9 Configuration

Farm layout is defined in `robot_brain/config/farm_locations.json`. Each location has x, y, yaw (map frame coordinates from SLAM) and a type (base, crop, utility). The `patrol_order` array defines the sequence of waypoints to visit.

```
{
  "locations": {
    "home":      {"x": 0.0, "y": 0.0, "yaw": 0.0, "type": "base"},
    "row_1":     {"x": 2.0, "y": 0.0, "yaw": 0.0, "type": "crop"},
    "tomato_section": {"x": 4.0, "y": 0.0, "yaw": 0.0, "type": "crop"}
  },
  "patrol_order": ["row_1", "row_2", "tomato_section"]
}
```

All thresholds and timing parameters are configurable via `robot_brain/config/farm_brain_config.yaml` or launch arguments.

10 Launch

```
# Full pipeline (camera + YOLO + STT + LLM + brain)
ros2 launch robot_brain farm_brain.launch.py

# Brain node only (sensors already running)
ros2 run robot_brain farm_brain

# With Isaac Sim model
ros2 launch robot_brain farm_brain.launch.py \
  isaac_model_path:=/path/to/policy.engine

# Custom farm layout
ros2 launch robot_brain farm_brain.launch.py \
  farm_locations_file:=/path/to/locations.json
```

11 Data Logging

After each complete patrol cycle, all waypoint measurements are appended to a JSON log file (default:

~/farm_brain_log.json). Each entry contains:

- Patrol timestamp
- Per-waypoint: soil moisture, pH, temp, humidity, CO2, UV, lux, rain, GPS, detected diseases, and action taken

This data can be used for trend analysis, ML training, or farm management dashboards.

12 All ROS 2 Nodes in the System

Complete list of ROS 2 nodes across all three platforms:

12.1 Jetson Orin Nano

Node	Package	Purpose
farm_brain	robot_brain	Autonomous farm orchestrator (this doc)
robot_brain	robot_brain	LLM text generation (Llama 3.2)
llm_node	llm_node	LLM intent parsing (Gemini / Llama)
stt_node	stt_node	Speech-to-text (Faster-Whisper GPU)
detection_node	yolov8_ros	YOLOv8m TensorRT + plant disease
camera	realsense2_camera	Intel RealSense D435 RGB-D
lidar	ldlidar_stl_ros2	LD-19 2D LiDAR driver
slam_toolbox	slam_toolbox	SLAM / localization
robot_state_publisher	robot_state_publisher	TF from URDF

12.2 Raspberry Pi 5

Node	Package	Purpose
tts_elevenlabs_node	tts_elevenlabs	Text-to-speech (ElevenLabs)
robot_face_display	llm_display	Animated robot face (PyQt5)
dashboard_display	llm_display	Sensor dashboard display
audio_monitor_node	speaker_monitor	Audio playback state
bno055_imu_node	bno055_imu	9-DOF IMU driver (I2C)

12.3 Raspberry Pi 4

Node	Package	Purpose
mecanum_driver	mecanum_driver	4x mecanum wheel control (serial)
motor_node	motor_control	GPIO motor + encoder driver
encoder_node	motor_control	4x rotary encoder reader
soil_moisture_node	soil_moisture	Soil moisture (Arduino serial)
rain_sensor_node	rain_sensor	Raindrop sensor (Arduino serial)
bmp180_node	bmp180_pressure	BMP180 pressure/temp (I2C)
gps_gt_u7_node	gps_gt_u7	GPS module (serial NMEA)
dht11_node	dht11 [placeholder]	DHT11 temp/humidity (GPIO)
ph_sensor_node	ph_sensor [placeholder]	pH sensor (Arduino ADC)
uv_sensor_node	uv_sensor [placeholder]	UV sensor (I2C/analog)
lux_sensor_node	lux_sensor [placeholder]	Ambient light (I2C)

gas_sensor_node	gas_sensor [placeholder]	MQ-135 CO2 (Arduino ADC)
-----------------	--------------------------	--------------------------

Nodes marked [placeholder] have topic interfaces defined in the farm brain but their ROS 2 packages have not yet been implemented. The topic names and message types are standardised so that when the nodes are created, the farm brain will automatically pick up their data.